

Artelier

Estructura y Organización del Proyecto

Guía completa de la arquitectura de directorios, componentes y convenciones

Stack: Next.js 16 · TypeScript · Prisma · T3 | Mayo 2026

CONTENIDO

01 Raíz del proyecto	09 Hooks React — src/hooks/
02 CI/CD — .github/workflows	10 Estado global — src/stores/
03 Base de datos — prisma/	11 Tipos TypeScript — src/types/
04 Assets estáticos — public/	12 Middleware de seguridad
05 Rutas y páginas — src/app/	13 Internacionalización — src/i18n/
06 APIs REST — src/app/api/	14 Plantillas de email — emails/
07 Componentes — src/components/	15 Filosofía de organización
08 Servicios externos — src/lib/	

01 Raíz del proyecto

```
artelier/  
■■■ .env.example ← Variables de entorno documentadas (sin valores reales)  
■■■ auth.config.ts ← Configuración de Auth.js separada del resto  
■■■ next.config.ts ← Configuración de Next.js (dominios de imagen, etc.)  
■■■ tailwind.config.ts ← Tokens de diseño de Tinta y Lino (colores, fuentes)  
■■■ package.json ← Dependencias y scripts
```

Los archivos de configuración van en la raíz porque las herramientas (Next.js, Tailwind, TypeScript) los buscan ahí por convención del ecosistema.

POR QUÉ AUTH.CONFIG.TS ESTÁ SEPARADO

Auth.js v5 necesita importar la configuración desde el middleware, que corre en Edge Runtime — un entorno más limitado que Node.js que no puede importar el cliente de Prisma. Separar la config de auth evita este conflicto.

02 CI/CD — .github/workflows/

```
.github/workflows/
■■■ ci.yml ← Se ejecuta en cada PR: lint + tests + build
■■■ preview.yml ← Despliega en Vercel una URL de preview por cada rama
```

ci.yml actúa como guardián — bloquea merges si el código rompe algo antes de llegar a producción.

preview.yml genera una URL única por cada PR para que cualquier persona del equipo pueda ver los cambios en vivo antes de mergear a main, sin necesidad de levantar el proyecto localmente.

03 Base de datos — prisma/

```
prisma/
■■■ schema.prisma ← Define todas las tablas: User, Product, Order, Message...
■■■ migrations/ ← Historial de cambios en la BD (como git, pero para SQL)
■■■ seed.ts ← Datos de prueba (artesanos ficticios, productos de ejemplo)
```

Prisma es el ORM — la capa que traduce entre TypeScript y PostgreSQL. El `schema.prisma` es la **fuentes de verdad** de la base de datos: cualquier cambio en la estructura pasa primero por aquí.

PARA QUÉ SIRVE MIGRATIONS/

Cada vez que cambias el schema (añades una columna, renombas un campo) se genera automáticamente un archivo SQL en `migrations/`. Esto garantiza que producción, staging y el entorno local de cada desarrollador tengan exactamente la misma estructura. Es el equivalente a git, pero para la base de datos.

seed.ts permite a cualquier desarrollador nuevo levantar el proyecto con datos realistas con un solo comando: `npx prisma db seed`.

04 Assets estáticos — public/

```
public/
■■■ fonts/ ← The Girl Next Door, DM Sans (self-hosted para velocidad)
■■■ icons/ ← Favicon, PWA icons, apple-touch-icon
```

Los archivos en `public/` se sirven directamente sin pasar por el servidor de Next.js — máxima velocidad y sin procesamiento.

POR QUÉ SELF-HOST LAS FUENTES

Cargar fuentes desde Google Fonts añade una petición de red extra a un servidor externo antes de poder mostrar el texto. Al alojar las fuentes en nuestro propio servidor (mismo dominio), eliminamos esa latencia y mejoramos el LCP (Largest Contentful Paint), que es una métrica de velocidad clave para SEO.

05 Rutas y páginas — src/app/

Next.js usa **file-system routing**: cada carpeta dentro de `app/` es una ruta URL. Un archivo `page.tsx` dentro de una carpeta es la página que se muestra en esa URL.

Route Groups — carpetas con paréntesis (nombre)

```
src/app/
  (auth)/ → URLs: /login, /register
  (buyer)/ → URLs: /feed, /search, /product/[id]...
  (artisan)/ → URLs: /studio/dashboard, /studio/products...
  (shared)/ → URLs: /messages, /profile, /settings
  admin/ → URLs: /admin/dashboard, /admin/artisans
```

LA CLAVE: LOS PARÉNTESIS NO APARECEN EN LA URL

(buyer)/feed/page.tsx → la URL es /feed, no /(buyer)/feed. Los paréntesis son solo organización interna. Cada grupo tiene su propio layout.tsx, lo que permite que compradores y artesanos vean interfaces completamente diferentes sin duplicar código.

Grupo (buyer) — Experiencia del comprador

```
feed/page.tsx → Pantalla principal con ProductCards
search/page.tsx → Búsqueda con filtros por categoría/sello
product/[id]/page.tsx → Detalle de producto (el [id] es dinámico)
artisan/[id]/page.tsx → Perfil público del artesano
cart/page.tsx → Carrito de compra
checkout/page.tsx → Formulario de envío + pago Stripe
orders/page.tsx → Historial de pedidos con OrderStatusTimeline
```

Grupo (artisan)/studio — Panel del artesano

```
dashboard/page.tsx → Resumen: ventas, pedidos pendientes, mensajes
products/page.tsx → Lista de sus productos publicados
products/new/page.tsx → Formulario para subir producto nuevo
orders/page.tsx → Pedidos recibidos con ProcessUpdate
```

Archivos especiales de Next.js en src/app/

Archivo	Función
layout.tsx	Estructura HTML compartida por todas las páginas (fuentes, providers de React)
page.tsx	Landing pública de Artelier, accesible sin autenticación
not-found.tsx	Página 404 con diseño de Artelier
error.tsx	Página de error inesperado — captura errores de React antes de que rompan la app

```

src/app/api/
  ■■■ auth/[...nextauth]/route.ts ← Auth.js gestiona /api/auth/login, /session...
  ■■■ products/
    ■■■ route.ts ← GET /api/products · POST /api/products
    ■■■ [id]/route.ts ← GET · PUT · DELETE /api/products/123
  ■■■ orders/
    ■■■ route.ts ← GET /api/orders · POST /api/orders
    ■■■ [id]/route.ts ← GET · PATCH /api/orders/123
  ■■■ messages/route.ts ← GET · POST /api/messages (polling)
  ■■■ upload/route.ts ← POST /api/upload → Cloudinary
  ■■■ webhooks/stripe/route.ts ← Stripe notifica aquí los pagos completados

```

La estructura sigue el patrón REST estándar: cada recurso tiene su carpeta, `route.ts` = endpoint de la colección, `[id]/route.ts` = recurso específico.

POR QUÉ EL WEBHOOK DE STRIPE ES CRÍTICO

Cuando un comprador completa un pago, Stripe envía una notificación HTTP a este endpoint. Debe verificar la firma criptográfica de Stripe (para evitar fraude) y procesar con idempotencia — si Stripe reenvía el mismo evento dos veces por red, el pedido no se crea duplicado.

07 Componentes React — src/components/

```

src/components/
  ■■■ ui/ ← Componentes genéricos de shadcn/ui (Button, Card, Input...)
  ■■■ artisan/
    ■■■ ArtisanHeader.tsx ← Cabecera del perfil (foto, nombre, badges)
    ■■■ SealBadge.tsx ← Badge/sello reutilizable (perfil y producto)
  ■■■ product/
    ■■■ ProductCard.tsx ← Tarjeta del feed (foto, precio, sello esquina)
  ■■■ order/
    ■■■ OrderStatusTimeline.tsx ← Línea de tiempo visual del estado del pedido
  ■■■ messaging/
    ■■■ ProcessUpdate.tsx ← Mensaje de actualización en el chat artesano
  ■■■ shared/
    ■■■ BottomNav.tsx ← Navegación inferior mobile (ambos roles)

```

POR QUÉ ORGANIZACIÓN FEATURE-FIRST

Los componentes se agrupan por dominio de negocio, no por tipo técnico. `ArtisanHeader.tsx` vive junto a `SealBadge.tsx` porque cuando tocas el perfil del artesano, editas los dos. No hay que buscar entre carpetas `'containers/'`, `'views/'`, `'presentational/'` — todo lo del artesano está en `artisan/`.

`ui/` es la excepción: aloja los componentes genéricos de shadcn/ui que no pertenecen a ningún dominio específico — son las "piezas de Lego" que usan todos los demás componentes.

08 Servicios externos — src/lib/

```
src/lib/
■■■ auth.ts ← Instancia configurada de Auth.js (providers, callbacks, RBAC)
■■■ db.ts ← Cliente Prisma singleton (una sola conexión a PostgreSQL)
■■■ stripe.ts ← Cliente Stripe inicializado con la API key
■■■ cloudinary.ts ← Función para subir y transformar imágenes
■■■ resend.ts ← Cliente Resend para enviar emails transaccionales
■■■ rate-limit.ts ← Upstash Redis para limitar requests por IP
■■■ utils.ts ← Funciones de utilidad (formatear precio en cents, etc.)
```

Cada archivo es un **singleton** — se instancia una sola vez y se reutiliza en toda la aplicación.

POR QUÉ DB.TS NECESITA SER SINGLETON

En desarrollo, Next.js recarga los módulos frecuentemente al detectar cambios. Sin el patrón singleton, cada recarga crearía una nueva conexión a PostgreSQL. En pocos minutos habrías saturado el pool de conexiones de la base de datos (Neon serverless tiene límite). El singleton garantiza que solo existe una conexión activa.

09 Custom React Hooks — src/hooks/

```
src/hooks/
■■■ useMessages.ts ← Polling cada 3s + pausa con Page Visibility API
■■■ useCart.ts ← Leer y modificar el carrito desde Zustand
```

Los hooks encapsulan lógica con estado que se comparte entre múltiples componentes.

useMessages contiene toda la complejidad del sistema de mensajería: el intervalo para hacer polling cada 3 segundos, la lógica para pausar automáticamente cuando la pestaña está en segundo plano (Page Visibility API), y la limpieza al desmontar el componente.

10 Estado global del cliente — src/stores/

```
src/stores/
■■■ cart.ts ← Estado del carrito: items, cantidades, total (Zustand)
```

REGLA ARQUITECTÓNICA: ZUSTAND SOLO PARA ESTADO UI

Zustand solo se usa para estado que no necesita persistirse en el servidor. El carrito es el único caso en V1: necesita sobrevivir la navegación entre páginas, pero no hace falta guardarlo en base de datos hasta el momento del checkout. Todo lo demás (sesión, datos de productos, pedidos) viene del servidor vía React Server Components o fetch.

11 Tipos TypeScript — src/types/

```
src/types/
■■■ api.ts ← Tipos del formato de respuesta: { data } / { error: { code, message } }
■■■ index.ts ← Re-exporta tipos de Prisma + tipos de dominio propios
```

api.ts define el contrato entre frontend y backend. Toda respuesta de la API tiene exactamente este formato, lo que hace imposible olvidar manejar errores.

index.ts re-exporta los tipos generados automáticamente por Prisma (User, Product, Order...) para que cualquier archivo pueda importarlos desde `@/types` en lugar de desde `@prisma/client`. Si en el futuro cambias el ORM, solo cambia este archivo.

12 Middleware de seguridad — src/middleware.ts

```
src/middleware.ts ← Se ejecuta ANTES de cada request, en el CDN de Vercel
```

Un solo archivo en la raíz de src/. Corre en **Edge Runtime** — en los servidores del CDN, antes de que Next.js cargue ninguna página.

Regla	Condición	Acción
Rutas protegidas	Acceso a /studio/* sin sesión	Redirige a /login
Control de roles	Acceso a /admin/* sin rol admin	Redirige a /feed
Redirección lógica	Acceso a /login con sesión activa	Redirige a /feed

POR QUÉ EN EL EDGE Y NO EN CADA PÁGINA

Si la protección estuviera en cada page.tsx, bastaría un error de programación en una página para dejar una ruta desprotegida. El middleware es la primera línea de defensa — corre antes de cualquier código de la aplicación, a nivel de red. Al ser Edge Runtime, añade solo ~1ms de latencia.

13 Internacionalización — src/i18n/

```
src/i18n/
  messages/
    es.json ← Todos los textos en castellano (completo desde V1)
    gl.json ← Todos los textos en gallego (completo en V3)
  config.ts ← Configuración de next-intl (idiomas, locale fallback)
```

POR QUÉ I18N DESDE EL PRIMER DÍA

Añadir internacionalización después de tener 200 textos escritos directamente en los componentes es un refactor costoso y propenso a errores. Empezar con es.json desde el principio tiene coste casi cero. Cuando llegue V3 con el gallego, solo habrá que traducir el JSON — ningún componente necesita modificarse.

14 Plantillas de email — emails/

```
emails/
  OrderConfirmation.tsx ← 'Tu pedido está confirmado' con detalle de productos
  PasswordReset.tsx ← Email con enlace de recuperación de contraseña
```

Escritas como componentes React usando **react-email**, que permite diseñarlos con preview en el navegador y estilos inline compatibles con todos los clientes de correo. Los envía el servicio **Resend**.

Están fuera de src/ porque no son componentes Next.js — son plantillas que solo usa el servidor cuando necesita notificar a un usuario. Mantenerlas separadas evita que se importen accidentalmente en el cliente.

15 Filosofía de organización: co-localización

El principio que une toda la estructura es la **co-localización**: el código que cambia junto, vive junto.

Convención	Se usa en	Ejemplo
------------	-----------	---------

PascalCase	Componentes, tipos, modelos	ProductCard, OrderStatus
camelCase	Funciones, variables, hooks	useMessages, formatPrice
kebab-case	Directorios, URLs, archivos no-componente	order-confirmation, /studio/products
SCREAMING_SNAKE	Constantes, valores de enum	ORDER_STATUS, MAX_IMAGES

REGLA DE MONTOS MONETARIOS

Todos los precios y montos se almacenan y operan en céntimos (enteros), nunca como decimales. €12.50 = 1250. Esto evita errores de punto flotante que en dinero son inaceptables ($0.1 + 0.2 = 0.30000000000000004$ en JavaScript).

LA VENTAJA PRÁCTICA

Si alguien te dice 'toca todo lo del perfil del artesano', sabes exactamente dónde ir: `src/app/(artisan)/`, `src/components/artisan/`, `src/app/api/artisan/`. No hay que explorar el proyecto entero ni adivinar dónde está cada cosa.